Primer: Sieve Estimation of Structural Models (SEES)

econirl package documentation

1 Problem and Motivation

The nested fixed point algorithm (NFXP) of **?** solves the Bellman equation to machine precision at every candidate parameter vector, costing $O(|\mathcal{S}|^2)$ per inner iteration. The CCP estimator of **?** eliminates the inner loop but requires an $O(|\mathcal{S}|^3)$ matrix inversion. Both methods scale with the full state space. When $|\mathcal{S}|$ reaches hundreds or thousands of states, as in multi-component replacement models or empirical games, the computational cost becomes prohibitive.

? propose the Sieve-based Efficient Estimator for Structural models (SEES). The value function is approximated by a small set of deterministic basis functions, and the Bellman equation is imposed as a penalty rather than a constraint. This produces a single unconstrained optimization over the structural parameters θ and basis coefficients α, with computational cost $O(K)$ independent of $|\mathcal{S}|$. No inner loop is solved. No neural network is trained. When $K = |\mathcal{S}|$ and the penalty weight goes to infinity, SEES reduces to the MPEC estimator of **?**.

2 Model and Notation

Table 1: Common DDC notation shared across all primers.

Symbol	Name	Definition
$s \in \mathcal{S}$	State	Discrete or continuous
$a \in \mathcal{A}$	Action	Finite: $\lvert\mathcal{A}\rvert = J$
s'	Next state	Drawn from $P(s' \mid s, a)$
$\beta \in [0, 1)$	Discount factor	
$\sigma > 0$	Scale parameter	Logit noise scale (= 1 by convention)
$\theta \in \Theta \subset \mathbb{R}^{d_\theta}$	Structural parameters	Unknown
$\phi(s, a)$	Feature vector	Known functions of (s, a), dimension K
$u(s, a; \theta)$	Flow utility	$\theta^\top \phi(s, a)$ (linear-in-parameters)
$P(s' \mid s, a)$	Transition kernel	Known or estimated from data
$Q(s, a)$	Choice-specific value	$u(s, a; \theta) + \beta \sum_{s'} P(s'\lvert s, a)\, V(s')$
$V(s)$	Integrated value	$\sigma \log \sum_a \exp(Q(s, a)/\sigma)$
$\pi(a \mid s)$	Choice probability	$\exp\big((Q(s, a) - V(s))/\sigma\big)$
τ	Trajectory	$(s_0, a_0, s_1, a_1, \ldots)$ from one agent
$\mathcal{D}$	Panel data	Collection of N observed trajectories

Symbol	Name	Definition
$\Psi(s)$	Sieve basis	Row vector of K basis functions evaluated at s
$\alpha \in \mathbb{R}^K$	Basis coefficients	Weights on the sieve basis
$\omega > 0$	Penalty weight	Controls strength of Bellman enforcement
K	Basis dimension	Number of sieve functions ($K \ll \lvert\mathcal{S}\rvert$)

The sieve approximation replaces $V(s)$ with

$$V(s;\alpha) = \Psi(s)^\top \alpha, \tag{1}$$

where $\Psi(s) = (\psi_1(s), \ldots, \psi_K(s))^\top$ is a deterministic basis. For Fourier bases, $\Psi(s) = (1, \cos(\pi\tilde{s}), \sin(\pi\tilde{s}), \cos(2\pi\tilde{s}), \ldots$ with $\tilde{s} \in [0,1]$. Substituting into the Bellman equation gives approximate action values $Q(s,a;\theta,\alpha) = \theta^\top \phi(s,a) + \beta\,(P_a\Psi)^\top \alpha$, where $P_a\Psi$ is precomputed once per action.

The penalized criterion jointly maximizes the log-likelihood and penalizes Bellman violations:

$$\max_{\theta,\alpha} \sum_{i=1}^{N} \log \pi(a_i \mid s_i;\theta,\alpha) \;-\; \omega \sum_{s} \big(V(s;\alpha) - T_\sigma(V;\theta)(s)\big)^2. \tag{2}$$

3 Core Results

Theorem 3 (Asymptotic Normality and Efficiency). If $\omega_n/n^2 \to \infty$ and regularity conditions P1–P9 hold, the SEES estimator satisfies

$$\sqrt{n}(\tilde{\theta}_n - \theta_0) \xrightarrow{d} N(0,\Sigma), \qquad \Sigma = -V_{\theta_0}^{-1},$$

attaining the Cramér-Rao lower bound. SEES is asymptotically efficient despite never solving the Bellman equation.

Corollary 3.1 (Schur Complement SEs). The joint Hessian over (θ,α) has a block structure. The marginal Hessian for θ is the Schur complement

$$\tilde{H}_\theta = H_{\theta\theta} - H_{\theta\alpha}\, H_{\alpha\alpha}^{-1}\, H_{\alpha\theta}. \tag{3}$$

Standard errors from $(-\tilde{H}_\theta)^{-1}$ marginalize out the basis coefficients and give the correct Fisher information for the structural parameters. Unlike NNES, which achieves a block-diagonal information matrix via the zero Jacobian property, SEES requires the full Schur complement because the off-diagonal blocks $H_{\theta\alpha}$ are generally nonzero.

Projection bias. If the value function has sharp discontinuities that the basis cannot represent, the projection bias contaminates $\hat{\theta}$. The paper recommends increasing K until parameter estimates stabilize. Smooth value functions are well-approximated by Fourier or polynomial bases at moderate K.

4 Estimation Algorithm

Algorithm 1: SEES: Joint Algorithm (**?**)

Input: Panel $\{(s_i, a_i)\}_{i=1}^N$, features $\phi(s, a)$, transitions $P(s'|s, a)$, discount β, scale σ, basis type, dimension K, penalty ω

`// Step 1: Build sieve basis`
$\Psi \leftarrow$ basis matrix of shape $(|\mathcal{S}|, K)$
Normalize states to $[0, 1]$ (Fourier) or $[-1, 1]$ (polynomial)

`// Step 2: Precompute continuation values`
for *each action* a **do**
  $\mathbb{E}_a[\Psi] \leftarrow P_a \cdot \Psi$ `// shape` $(|\mathcal{S}|, K)$`, one matrix multiply`
end

`// Step 3: Define penalized criterion (Eq. 3)`
$x = [\theta, \alpha] \in \mathbb{R}^{d_\theta + K}$
$f(x) = \sum_i \log \pi(a_i|s_i; \theta, \alpha) - \omega \sum_s (V(s; \alpha) - T_\sigma(V; \theta)(s))^2$

`// Step 4: Joint L-BFGS-B optimization`
$\hat{x} \leftarrow \arg\max_x f(x)$ `// single call, autodiff gradient`
$\hat{\theta}, \hat{\alpha} \leftarrow \hat{x}[: d_\theta],\ \hat{x}[d_\theta :]$

`// Step 5: Standard errors via Schur complement (Eq. 4)`
$H \leftarrow$ numerical Hessian of f at $\hat{x}$
$\tilde{H}_\theta \leftarrow H_{\theta\theta} - H_{\theta\alpha} H_{\alpha\alpha}^{-1} H_{\alpha\theta}$
$\mathrm{SE}(\hat{\theta}) \leftarrow \sqrt{\mathrm{diag}(-\tilde{H}_\theta^{-1})}$

Output: $\hat{\theta}$, V^*, π^*, $\mathrm{SE}(\hat{\theta})$, log-likelihood

Implementation. The entire estimation is a single L-BFGS-B call over the $(d_\theta + K)$-dimensional vector. JAX autodiff computes the gradient. The precomputed $P_a \Psi$ matrices make continuation value evaluation a matrix-vector multiply, avoiding any inner loop.

5 Results

Table 2: 2-component bus engine (900 states, $\beta = 0.9999$, 10,000 obs). SEES uses a Fourier basis with $K = 8$.

	True	NFXP	SEES
$replacement_cost$	3.0000	2.9274	3.0345
SE	—	0.1115	0.0946
$operating_cost$	0.0010	-0.0477	0.0925
SE	—	0.0460	0.0306
$quadratic_cost$	0.0005	0.0230	-0.0390
SE	—	0.0199	0.0238
Log-lik	—	-1948.7	-1949.0
Time (s)	—	397.2	2.5

We run SEES and NFXP on a 2-component bus engine with 900 total states (30 mileage bins per component), discount $\beta = 0.9999$, and 10,000 observations. SEES uses a Fourier basis with $K = 8$

terms. This environment tests the scaling argument from **?**: with $|\mathcal{S}| = 900$, NFXP must solve a 900×900 Bellman fixed point at each parameter guess, while SEES optimizes over $d_\theta + 8 = 11$ unknowns regardless of $|\mathcal{S}|$.

SEES achieves a log-likelihood within 0.36 nats of NFXP while running $160\times$ faster (2.5s vs 397.2s). The speedup comes entirely from eliminating the inner fixed-point iteration: at $\beta = 0.9999$, the Bellman contraction rate is $(1 - \beta)$, requiring tens of thousands of value iteration steps per NFXP inner call. SEES bypasses this entirely.

SEES occupies a middle ground between MPEC (exact V, $O(|\mathcal{S}|)$ unknowns) and NNES (neural V, valid SEs via Neyman orthogonality). The basis functions are deterministic and require no training, making the optimization smooth and well-conditioned. The trade-off is projection bias when V is not well-represented by the basis. NNES avoids this with a universal approximator, at the cost of alternating optimization and sensitivity to initialization.

Code. The companion script `sees_run.py` generates Table 2. To regenerate: `python papers/econirl_package/p`